ОТЧЁТ

по лабораторной работе №16

«Веб-приложение на Django»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться описывать предметную область моделями Django, хранить её в базе и показывать абонентов, тарифы, услуги, подключения и платежи на страницах.

2. Постановка индивидуального задания

Вариант 20. Сущности Client, Tariff, Service, Connection и Payment. Функциональность: абоненты, тарифы, услуги, подключения и платежи.

3. Краткие теоретические сведения

Модель Django — класс, по которому строится таблица. ForeignKey задаёт связь «многие к одному», ManyToManyField — связь подключения и услуг. Миграция фиксирует эту схему в базе. Представление читает QuerySet и передаёт его в шаблон.

POST в Django защищён CSRF-токеном: форма содержит скрытое поле, без него сервер отклоняет чужую отправку. После изменения данных представление перенаправляет на карточку абонента.

4. Описание программы

У клиента есть баланс и телефон, у тарифа — скорость и цена, у услуги — своя цена. Подключение связывает клиента с тарифом и набором услуг и считает ежемесячную сумму, если статус «активно». Платёж увеличивает баланс.

Начальные данные: Иванов на «Домашнем 100» с услугой ТВ и Петрова на корпоративном тарифе с белым IP. Проверка на сервере приняла платёж 500 рублей с комментарием «оплата сентября»: баланс Иванова стал 1700.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — billing__init__.py

Листинг — billing\admin.py

```
from django.contrib import admin

from .models import Client, Connection, Payment, Service, Tariff


admin.site.register(Client)
admin.site.register(Tariff)
admin.site.register(Service)
admin.site.register(Connection)
admin.site.register(Payment)
```

Листинг — billing\apps.py

```
from django.apps import AppConfig



class BillingConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "billing"
    verbose_name = "Биллинг провайдера"
```

Листинг — billing\models.py

```
from django.db import models



class Client(models.Model):
    name = models.CharField("имя", max_length=120)
    phone = models.CharField("телефон", max_length=20)
    balance = models.DecimalField("баланс", max_digits=10, decimal_places=2, default=0)

    def __str__(self):
        return self.name



class Tariff(models.Model):
    name = models.CharField("название", max_length=120)
    speed = models.PositiveIntegerField("скорость, Мбит/с")
    price = models.DecimalField("цена", max_digits=10, decimal_places=2)

    def __str__(self):
        return self.name



class Service(models.Model):
    name = models.CharField("услуга", max_length=120)
    price = models.DecimalField("цена", max_digits=10, decimal_places=2)

    def __str__(self):
        return self.name



class Connection(models.Model):
    STATUS = [("active", "активно"), ("suspended", "приостановлено"), ("disabled", "отключено")]
    client = models.ForeignKey(Client, on_delete=models.CASCADE, related_name="connections")
    tariff = models.ForeignKey(Tariff, on_delete=models.PROTECT, related_name="connections")
    services = models.ManyToManyField(Service, blank=True, related_name="connections")
    status = models.CharField("статус", max_length=20, choices=STATUS, default="active")
    started = models.DateField("дата подключения", auto_now_add=True)

    def monthly_total(self):
        extra = sum(service.price for service in self.services.all())
        if self.status != "active":
            return 0
        return self.tariff.price + extra



class Payment(models.Model):
    client = models.ForeignKey(Client, on_delete=models.CASCADE, related_name="payments")
    amount = models.DecimalField("сумма", max_digits=10, decimal_places=2)
```

```
    comment = models.CharField("комментарий", max_length=200, blank=True)
    created = models.DateTimeField(auto_now_add=True)
```

Листинг — billing\urls.py

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.index, name="index"),
    path("clients/<int:client_id>/", views.client_detail, name="client"),
]
```

Листинг — billing\views.py

```
from decimal import Decimal

from django.shortcuts import get_object_or_404, redirect, render

from .models import Client, Connection, Payment, Service, Tariff



def index(request):
    return render(request, "billing/index.html", {
        "clients": Client.objects.all(),
        "tariffs": Tariff.objects.all(),
        "services": Service.objects.all(),
    })



def client_detail(request, client_id):
    client = get_object_or_404(Client, pk=client_id)
    if request.method == "POST":
        action = request.POST.get("action")
        if action == "connect":
            tariff = get_object_or_404(Tariff, pk=request.POST["tariff_id"])
            connection = Connection.objects.create(client=client, tariff=tariff)
            for service_id in request.POST.getlist("services"):
                connection.services.add(service_id)
        elif action == "pay":
            amount = Decimal(request.POST["amount"].replace(",", "."))
            Payment.objects.create(client=client, amount=amount, comment=request.POST.get("comment", ""))
            client.balance += amount
            client.save()
        elif action == "status":
            connection = get_object_or_404(Connection, pk=request.POST["connection_id"], client=client)
            connection.status = request.POST["status"]
            connection.save()
        return redirect("client", client_id=client.id)
    return render(request, "billing/client.html", {
        "client": client,
        "tariffs": Tariff.objects.all(),
        "services": Service.objects.all(),
        "connections": client.connections.select_related("tariff").prefetch_related("services"),
        "payments": client.payments.order_by("-created"),
    })
```

Листинг — manage.py

```
#!/usr/bin/env python
import os
import sys



def main():
    os.environ.setdefault("DJANGO_SETTINGS_MODULE", "provider_site.settings")
    from django.core.management import execute_from_command_line
    execute_from_command_line(sys.argv)



if __name__ == "__main__":
    main()
```

Листинг — provider_site__init__.py

Листинг — provider_site\settings.py

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = "lab16-provider-variant-20"
DEBUG = True
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "billing",
]
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
]
ROOT_URLCONF = "provider_site.urls"
TEMPLATES = [{
    "BACKEND": "django.template.backends.django.DjangoTemplates",
    "DIRS": [BASE_DIR / "templates"],
    "APP_DIRS": True,
    "OPTIONS": {"context_processors": [
        "django.template.context_processors.request",
        "django.contrib.auth.context_processors.auth",
        "django.contrib.messages.context_processors.messages",
    ]},
}]
WSGI_APPLICATION = "provider_site.wsgi.application"
DATABASES = {"default": {"ENGINE": "django.db.backends.sqlite3", "NAME": BASE_DIR / "db.sqlite3"}}
LANGUAGE_CODE = "ru-ru"
TIME_ZONE = "Europe/Moscow"
USE_I18N = True
USE_TZ = True
STATIC_URL = "static/"
DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

Листинг — provider_site\urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("billing.urls")),
]
```

Листинг — provider_site\wsgi.py

```
import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "provider_site.settings")
application = get_wsgi_application()
```

Листинг — seed.py

```
"""Начальные данные провайдера."""

import os

import django

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "provider_site.settings")
django.setup()
```

```
from billing.models import Client, Connection, Payment, Service, Tariff


def main():
    if Client.objects.exists():
        print("Данные уже есть")
        return
    home = Tariff.objects.create(name="Домашний 100", speed=100, price=500)
    corp = Tariff.objects.create(name="Корпоративный 500", speed=500, price=2500)
    ip = Service.objects.create(name="Белый IP", price=200)
    tv = Service.objects.create(name="ТВ", price=250)
    ivan = Client.objects.create(name="Иванов", phone="79001112233", balance=1200)
    anna = Client.objects.create(name="Петрова", phone="79004445566", balance=300)
    first = Connection.objects.create(client=ivan, tariff=home, status="active")
    first.services.add(tv)
    second = Connection.objects.create(client=anna, tariff=corp, status="active")
    second.services.add(ip)
    Payment.objects.create(client=ivan, amount=1200, comment="аванс")
    print("Тестовые абоненты созданы")


if __name__ == "__main__":
    main()
```

Листинг — templates\billing\client.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>{{ client.name }}</title>
  <style>
    body { font-family: Segoe UI, sans-serif; margin: 32px; background: #f6f8fb; }
    table { border-collapse: collapse; background: white; width: 100%; margin-bottom: 16px; }
    th, td { border-bottom: 1px solid #e5ebf2; padding: 8px; text-align: left; }
    form { background: white; padding: 12px; margin: 12px 0; }
    a { color: #0b4f8a; }
  </style>
</head>
<body>
  <p><a href="{% url 'index' %}">К списку</a></p>
  <h1>{{ client.name }}</h1>
  <p>Телефон {{ client.phone }}, баланс {{ client.balance }} руб.</p>
  <h2>Подключения</h2>
  <table>
    <tr><th>Тариф</th><th>Услуги</th><th>Статус</th><th>К оплате</th></tr>
    {% for connection in connections %}
    <tr>
      <td>{{ connection.tariff.name }}</td>
      <td>{% for service in connection.services.all %}{{ service.name }} {% empty %}нет{% endfor %}</td>
      <td>{{ connection.get_status_display }}</td>
      <td>{{ connection.monthly_total }} руб.</td>
    </tr>
    {% empty %}
    <tr><td colspan="4">Подключений нет</td></tr>
    {% endfor %}
  </table>
  <form method="post">
    {% csrf_token %}
    <input type="hidden" name="action" value="connect">
    <h3>Подключить тариф</h3>
    <select name="tariff_id">{% for tariff in tariffs %}<option value="{{ tariff.id }}">{{ tariff.name
}}</option>{% endfor %}</select>
    {% for service in services %}
    <label><input type="checkbox" name="services" value="{{ service.id }}"> {{ service.name }}</label>
    {% endfor %}
    <button>Подключить</button>
  </form>
  <h2>Платежи</h2>
  <table>
    <tr><th>Сумма</th><th>Комментарий</th></tr>
    {% for payment in payments %}
    <tr><td>{{ payment.amount }}</td><td>{{ payment.comment }}</td></tr>
    {% empty %}<tr><td colspan="2">Платежей нет</td></tr>{% endfor %}
  </table>
```

```
  <form method="post">
    {% csrf_token %}
    <input type="hidden" name="action" value="pay">
    <input name="amount" placeholder="Сумма" required>
    <input name="comment" placeholder="Комментарий">
    <button>Принять платёж</button>
  </form>
</body>
</html>
```

Листинг — templates\billing\index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Интернет-провайдер</title>
  <style>
    body { font-family: Segoe UI, sans-serif; margin: 32px; background: #f6f8fb; color: #1c2430; }
    a { color: #0b4f8a; }
    table { border-collapse: collapse; background: white; width: 100%; }
    th, td { border-bottom: 1px solid #e5ebf2; padding: 8px 10px; text-align: left; }
    h1 { margin-bottom: 4px; }
  </style>
</head>
<body>
  <h1>Интернет-провайдер</h1>
  <p>Абоненты, тарифы, услуги, подключения и платежи.</p>
  <h2>Абоненты</h2>
  <table>
    <tr><th>Имя</th><th>Телефон</th><th>Баланс</th></tr>
    {% for client in clients %}
    <tr>
      <td><a href="{% url 'client' client.id %}">{{ client.name }}</a></td>
      <td>{{ client.phone }}</td>
      <td>{{ client.balance }} руб.</td>
    </tr>
    {% endfor %}
  </table>
  <h2>Тарифы</h2>
  <ul>
    {% for tariff in tariffs %}
    <li>{{ tariff.name }} — {{ tariff.speed }} Мбит/с, {{ tariff.price }} руб.</li>
    {% endfor %}
  </ul>
</body>
</html>
```

6. Примеры работы программы

Снимки сделаны с запущенного сайта: общий список и карточка абонента после платежа.

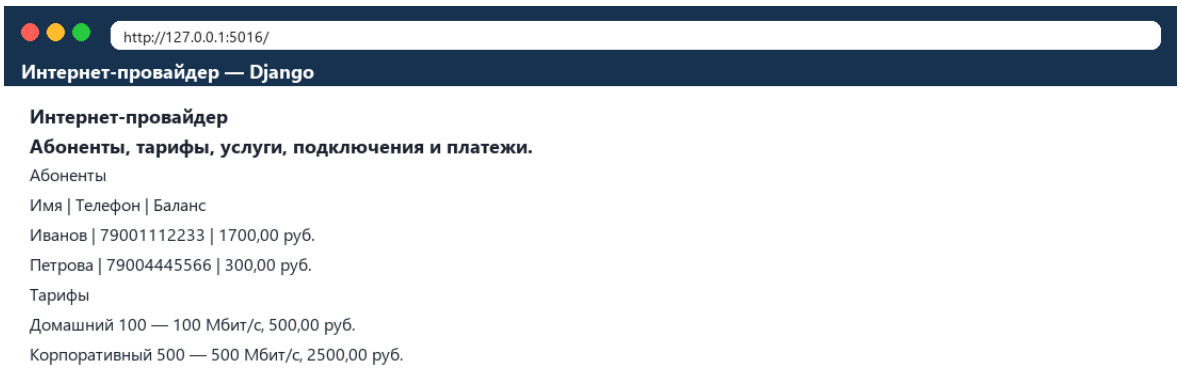


Рисунок 1 – Список абонентов и тарифов



Рисунок 2 – Карточка Иванова: подключение, услуги и платежи

7. Ответы на контрольные вопросы

1. Чем модель Django отличается от обычного класса?

По полям модели создаётся таблица, а объекты выбираются запросами ORM, например Client.objects.all().

2. Как связаны подключение и услуги?

ManyToManyField. Одна услуга может входить в разные подключения, у подключения может быть несколько услуг.

3. Зачем миграции?

Они переносят описание моделей в реальную схему SQLite. Без миграции таблиц ещё нет.

4. Что защищает CSRF?

Токен не даёт чужому сайту отправить платёж от имени открытой сессии пользователя.

5. Где считается сумма к оплате?

В методе Connection.monthly_total: цена тарифа плюс цены услуг, если подключение активно.

8. Вывод

В лабораторной работе №16 на Django собрана модель провайдера варианта 20: клиенты, тарифы, услуги, подключения и платежи хранятся в SQLite и показываются на страницах. Платёж, принятый через форму, увеличил баланс абонента и появился в истории.